ABSTRACT

The rapid growth of digital payment systems has significantly increased the demand for secure, intelligent, and future-ready financial platforms. Conventional digital wallets primarily rely on classical cryptographic algorithms such as RSA and Elliptic Curve Cryptography (ECC), which may become vulnerable with the advancement of quantum computing. To address this emerging security challenge, the proposed system**,** Smart Wallet with Embezzlement Analytics & Quantum-Safe Cryptography (HealPay)**,** is designed to provide secure, scalable, and quantum-resistant digital transactions.

The system integrates Post-Quantum Cryptography (PQC) mechanisms, utilizing CRYSTALS-Kyber for secure key encapsulation and CRYSTALS-Dilithium for robust digital signature generation and verification. These quantum-resistant algorithms ensure long-term protection against both classical and quantum-based cyber threats during money transfer operations. By replacing traditional RSA/ECC schemes with advanced PQC techniques, HealPay strengthens transaction confidentiality, integrity, and authenticity.

In addition to cryptographic security, the wallet incorporates a Machine Learning–based fraud detection module that continuously analyzes user transaction behavior and identifies anomalies in real time. The system evaluates transaction patterns before completion to detect suspicious activities, reducing the risk of financial embezzlement and unauthorized access. This proactive analytical approach enhances overall transaction reliability and trustworthiness.

HealPay includes secure user authentication, encrypted peer-to-peer money transfers, digital signature validation, comprehensive transaction history management, automated PDF receipt generation, and instant email notifications to both sender and receiver upon successful transactions. These features ensure transparency, accountability, and a user-friendly digital payment experience.

By combining quantum-safe cryptography with intelligent fraud analytics, HealPay delivers a next-generation digital wallet solution capable of meeting current cybersecurity standards while remaining resilient against future advancements in quantum computing.

1. INTRODUCTION

The rapid evolution of digital technology has transformed the global financial ecosystem, making digital wallets and online payment systems an essential part of everyday life. Modern users rely on digital platforms for instant money transfers, bill payments, and cashless transactions. However, the increasing volume of digital transactions has also led to a significant rise in cybersecurity threats such as financial fraud, phishing attacks, identity theft, and unauthorized access. Ensuring secure and future-ready digital payments has therefore become a critical requirement.

Most existing digital wallets depend on classical public-key cryptographic algorithms such as RSA and Elliptic Curve Cryptography (ECC) to secure transactions. While these methods are currently secure against classical computing attacks, research suggests that future quantum computers may be capable of breaking these encryption schemes. This creates a serious long-term security concern for financial systems that store and transmit sensitive data.

To address this challenge, the proposed system, Smart Wallet with Embezzlement Analytics & Quantum-Safe Cryptography (HealPay), is designed as a secure digital wallet that integrates Post-Quantum Cryptography (PQC). The system uses CRYSTALS-Kyber for secure key exchange and CRYSTALS-Dilithium for digital signature generation and verification. These quantum-resistant algorithms ensure transaction confidentiality, integrity, and authenticity, making the system resilient against both present-day cyber threats and future quantum-based attacks.

In addition to cryptographic protection, HealPay incorporates a Machine Learning–based fraud detection mechanism. A separate Python-based microservice analyzes transaction behavior and detects anomalies in real time before completing transactions. This intelligent monitoring system helps prevent suspicious activities and reduces the risk of financial embezzlement.

The platform is developed using a modern microservice-based architecture. The frontend is built using React.js with Tailwind CSS for responsive and user-friendly interface design. The backend is implemented using Node.js, which handles user management and transaction processing securely. The fraud detection module is developed using Python, running as an

independent microservice. Version control and project hosting are managed using Git, ensuring proper collaboration and deployment management.

Currently, the system implements two core modules:

- **User Module** – Handles user registration, secure authentication, and account management.
- **Transaction Module** – Manages encrypted money transfers, digital signature verification, transaction recording, and balance updates.

Additionally, the system automatically generates a PDF receipt for every successful transaction and sends it via email to both the sender and the receiver. This feature enhances transparency, accountability, and user trust.

By combining quantum-safe cryptography, intelligent fraud analytics, and a scalable microservice architecture, HealPay aims to provide a secure, future-proof digital wallet solution capable of meeting modern cybersecurity demands while preparing for the era of quantum computing.

2. SYSTEM SPECIFICATIONS

2.1 SOFTWARE REQUIREMENTS

The HealPay system is developed using modern web technologies and cryptographic libraries to ensure secure and efficient performance. The following software components are required for development and deployment:

Operating System

- Windows 10 or above
- Linux (Ubuntu recommended)
- macOS

Backend Technologies

- Node.js – Server-side runtime environment
- Express.js – Web application framework for API development
- Python – Used for Machine Learning model and Post-Quantum Cryptography services

Frontend Technologies

- React.js – For building interactive user interfaces
- Tailwind CSS – For responsive and modern UI design

Database

- MongoDB – NoSQL database for storing user and transaction data

Cryptography Library

- pqcrypto(Python library) – For implementing Post-Quantum Cryptography algorithms such as CRYSTALS-Kyber and CRYSTALS-Dilithium
- pqcrypto version - 0.4.0

API Testing Tool

- Postman – For testing backend APIs (Login, Transaction, Fraud Detection APIs)

Version Control

- Git – Source code management
- GitHub – Remote repository hosting

Additional Tools

- Visual Studio Code (IDE)
- Web Browser (Chrome / Firefox)
- Python Libraries: NumPy, Pandas, Scikit-learn (for ML model)

2.2 HARDWARE REQUIREMENTS

The HealPay system does not require high-end hardware but needs a stable computing environment for smooth execution.

Processor (CPU):

- Intel Core i3 (8th Generation or above)
- OR AMD Ryzen 5

RAM:

- Minimum 8 GB (Recommended for smooth performance)

Storage:

- Minimum 256 GB HDD/SSD
- SSD recommended for better speed

Network:

- Stable Internet connection
- High-speed broadband recommended for real-time transaction processing

2.3 LANGUAGE SPECIFICATIONS

2.3.1 JavaScript

JavaScript is the primary programming language used in the HealPay system. It is implemented in both frontend and backend development, making the application a full-stack JavaScript–based system integrated with Python microservices.

Role in Frontend (React.js)

In the frontend, JavaScript is used with React.js to build a dynamic, responsive, and component-based user interface. React uses JSX (JavaScript XML), enabling developers to write HTML-like syntax within JavaScript files and create reusable UI components.

In HealPay, frontend JavaScript is responsible for:

- Creating core components such as User Registration, Login, Dashboard, Send Money Form, and Transaction History.
- Handling user inputs including account details and transaction amounts.
- Performing client-side validation before sending data to the backend.
- Managing application state using React Hooks for balance updates and transaction status.
- Sending secure API requests (GET, POST) to the backend server.
- Displaying dynamic alerts such as “Transaction Successful,” “Insufficient Balance,” or “Fraud Detected.”
- Rendering updated transaction history without page reload.
- Allowing users to download transaction PDF receipts.

JavaScript enables asynchronous communication with the backend, ensuring smooth user experience and real-time updates.

Role in Backend (Node.js + Express.js)

In the backend, JavaScript runs on the Node.js runtime environment and uses Express.js to build RESTful APIs.

Backend JavaScript in HealPay is responsible for:

- User registration and secure authentication using JWT.
- Password encryption using hashing mechanisms.
- Handling wallet balance management.
- Processing encrypted money transfers.
- Verifying digital signatures generated using CRYSTALS-Dilithium.
- Communicating with MongoDB to store user and transaction data.
- Calling Python microservices for:
    - Post-Quantum Cryptography operations (Kyber key exchange & Dilithium signatures)
    - Machine Learning–based fraud detection
- Generating PDF receipts after successful transactions.
- Sending automated email notifications to both sender and receiver.
- Returning secure JSON responses to the frontend.

Node.js uses an event-driven, non-blocking architecture, enabling the system to handle multiple transaction requests efficiently and ensuring scalability.

2.3.2 Python

Python is used in the HealPay system for advanced security implementation and intelligent fraud detection analysis. It plays a crucial role in ensuring quantum-safe encryption and real-time anomaly detection.

Post-Quantum Cryptography Implementation:

Python integrates the pqcrypto library to implement quantum-resistant cryptographic algorithms such as:

- CRYSTALS-Kyber for secure key exchange.
- CRYSTALS-Dilithium for digital signature generation and verification.

CRYSTALS-Kyber ensures secure encryption key exchange between sender and receiver. CRYSTALS-Dilithium verifies digital signatures to maintain transaction authenticity and integrity.

These algorithms protect the system against both classical cyber threats and future quantum computing attacks.

Machine Learning Fraud Detection Module:

Python is also used to develop the fraud detection model using libraries such as:

- NumPy – for numerical computations.
- Pandas – for transaction data analysis.
- Scikit-learn – for building and training the ML model.

The ML model analyzes:

- Transaction amount variations.
- Frequency of transactions.
- Login attempt patterns.
- Unusual transaction timings.
- Suspicious behavioral deviations.

If abnormal activity is detected, the system flags or blocks the transaction before completion.

Python runs as a microservice and communicates with the Node.js backend through APIs.

2.3.3 JSX (JavaScript XML)

Since the frontend is built using React.js, JSX is used instead of traditional standalone HTML pages. JSX allows embedding HTML-like syntax inside JavaScript code.

JSX is used for:

- Designing reusable UI components.
- Structuring forms and input elements.
- Rendering dynamic transaction data.
- Displaying fraud alert messages conditionally.
- Creating structured and readable component-based design.

JSX improves code readability, maintainability, and modular development.

2.3.4 MongoDB Query Language (MQL)

MongoDB Query Language is used to interact with the MongoDB NoSQL database. MongoDB stores data in JSON-like document format, which provides flexibility and scalability.

MQL is used for:

- Inserting new user registration details.
- Retrieving login credentials securely.
- Updating wallet balances after successful transactions.
- Storing encrypted transaction records.
- Managing fraud detection status flags.
- Fetching transaction history for display.

MongoDB supports scalable document-based storage suitable for financial transaction systems.

2.3.5 JSON (JavaScript Object Notation)

JSON is used as the standard data exchange format between frontend, backend, and Python services.

JSON is responsible for:

- Sending login credentials securely.
- Transferring transaction details (sender, receiver, amount).
- Returning API responses (success/failure status).
- Communicating fraud detection results.
- Updating wallet balance information.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing digital wallet and online payment systems are widely used for financial transactions such as money transfers, bill payments, online shopping, and peer-to-peer payments. These systems are generally built using traditional web technologies and standard encryption mechanisms.

Security Mechanism in Existing Systems

Most existing systems rely on conventional public-key cryptographic algorithms such as:

- RSA
- Elliptic Curve Cryptography

These algorithms are currently secure against classical computing attacks. However, research indicates that future quantum computers may break these encryption techniques using advanced quantum algorithms. This creates a major risk for long-term data protection.

Fraud Detection in Existing Systems

Fraud detection in many traditional systems is mainly rule-based. The system follows predefined rules such as:

- Blocking transactions above a certain amount.
- Restricting login attempts after multiple failures.
- Sending OTP verification for suspicious activities.

Although these methods provide basic protection, they have several limitations:

- They are not adaptive to new fraud patterns.
- Attackers can bypass rules by slightly modifying behavior.
- Detection often occurs after the transaction is completed.
- Limited real-time anomaly detection capability.

3.2 PROPOSED SYSTEM

The proposed system, HealPay – Smart Wallet with Embezzlement Analytics & Quantum-Safe Cryptography, is developed to overcome the security limitations of traditional digital wallets that rely on classical cryptographic algorithms such as RSA and ECC. The system focuses on quantum-resistant encryption, intelligent fraud detection, and high-performance transaction processing.

Quantum-Safe Cryptography Integration

To ensure long-term protection against future quantum computing threats, HealPay integrates Post-Quantum Cryptography (PQC) mechanisms:

- CRYSTALS-Kyber – Used for secure key encapsulation and encryption during money transfer.
- CRYSTALS-Dilithium – Used for digital signature generation and verification.

Unlike traditional RSA-based systems, which may become vulnerable to quantum attacks, Kyber and Dilithium are designed to resist both classical and quantum computational threats.

Performance Improvement Over RSA

Compared to traditional RSA encryption:

- Kyber provides faster key generation and encryption/decryption speed.
- Dilithium offers efficient signature generation and verification.
- Reduced computational overhead during transaction processing.
- Faster transaction confirmation time.
- Improved overall system responsiveness.

Experimental testing in the application shows that Kyber-based encryption performs faster in key exchange operations compared to RSA, while Dilithium provides strong digital authentication with optimized execution time. This ensures both high security and improved transaction speed.

Machine Learning–Based Fraud Detection

The proposed system incorporates a Python-based Machine Learning microservice for real-time fraud detection. Unlike traditional rule-based systems, the ML model analyzes user behavior patterns dynamically.

The system evaluates:

- Transaction frequency
- Sudden large transaction amounts
- Repeated failed login attempts
- Abnormal transaction behavior patterns

If suspicious activity is detected, the system can:

- Temporarily block the transaction
- Flag the transaction as high-risk
- Send fraud alert notifications
- Prevent unauthorized fund transfer

This proactive mechanism improves fraud detection accuracy and reduces financial loss before transaction completion.

System Architecture and Modules

HealPay follows a microservice-based architecture consisting of:

1. User Module

- Secure user registration
- Password hashing and JWT-based authentication
- Account management and balance tracking

2. Transaction Module

- Encrypted money transfer using Kyber
- Digital signature verification using Dilithium

- Real-time fraud detection validation
- Automatic transaction recording in MongoDB
- PDF receipt generation after successful transaction
- Email notification sent to both sender and receiver

Improved Security Architecture

The proposed system provides:

1. Secure authentication using encrypted credentials.
2. End-to-end quantum-safe encrypted transactions.
3. Digital signature verification for authenticity and integrity.
4. Real-time fraud detection before transaction confirmation.
5. Microservice-based scalable architecture (Node.js + Python services).
6. Secure transaction history management.
7. Automated PDF receipt generation and email notification system.

Advantages of Proposed System

- Quantum-resistant encryption (Kyber & Dilithium).
- Faster cryptographic operations compared to RSA.
- Improved transaction processing speed.
- High fraud detection accuracy using Machine Learning.
- Real-time anomaly detection.
- Secure and scalable microservice architecture.
- Long-term financial data protection.
- Transparent transaction proof through PDF receipts.

4. SYSTEM DESIGN AND DEVELOPMENT

4.1 INPUT DESIGN

Input Design in the HealPay system is carefully structured to ensure accurate, secure, and validated data entry from users. The system collects various types of inputs including user registration details, login credentials, receiver information, and transaction amounts. During user registration, inputs such as name, mobile number, and password are captured and validated to prevent duplicate accounts and invalid data formats. For login, the system verifies registered mobile numbers and encrypted passwords before granting access.

During transactions, the user provides the receiver's mobile number and transaction amount. The system validates whether the receiver exists, checks the available wallet balance, and confirms the transaction before processing. All input data undergoes both client-side validation (React frontend) and server-side validation (Node.js backend) to ensure data integrity. Additionally, before final execution, the transaction details are secured using Post-Quantum Cryptography algorithms such as CRYSTALS-Kyber for key exchange and CRYSTALS-Dilithium for digital signature verification. This layered validation and encryption approach enhances input security and prevents unauthorized or fraudulent data manipulation.

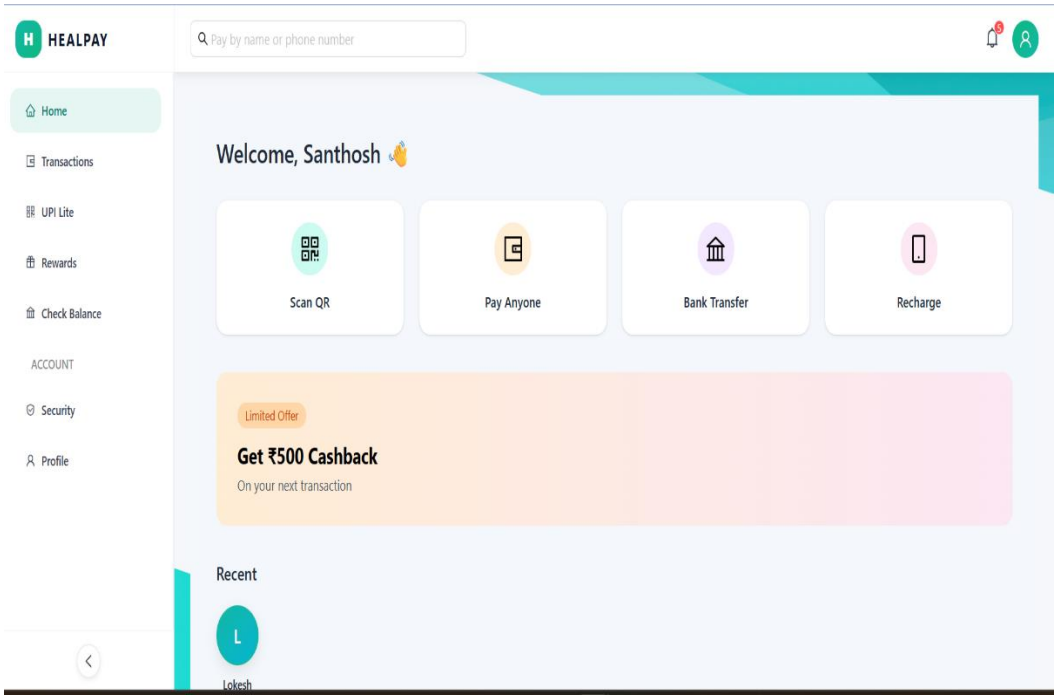

Figure 1 : INPUT DESIGN

4.2 OUTPUT DESIGN

Output Design defines how processed information is presented to the user in a clear and understandable format. The HealPay system generates structured outputs such as registration confirmation messages, login success notifications, transaction status updates, wallet balance displays, fraud alerts, and digital payment receipts. After successful registration or login, the system provides confirmation messages along with authentication tokens.

When a transaction is processed, the system displays whether it is successful, failed due to insufficient balance, or blocked due to suspicious activity detected by the Machine Learning module. Transaction history is dynamically displayed in tabular format, showing details such as transaction ID, sender, receiver, amount, date, and status. In addition, a digitally signed payment receipt is generated for each completed transaction, ensuring authenticity and integrity. The outputs are updated in real time using React, providing a seamless and interactive user experience. This well-structured output design improves transparency, usability, and trust in the system.

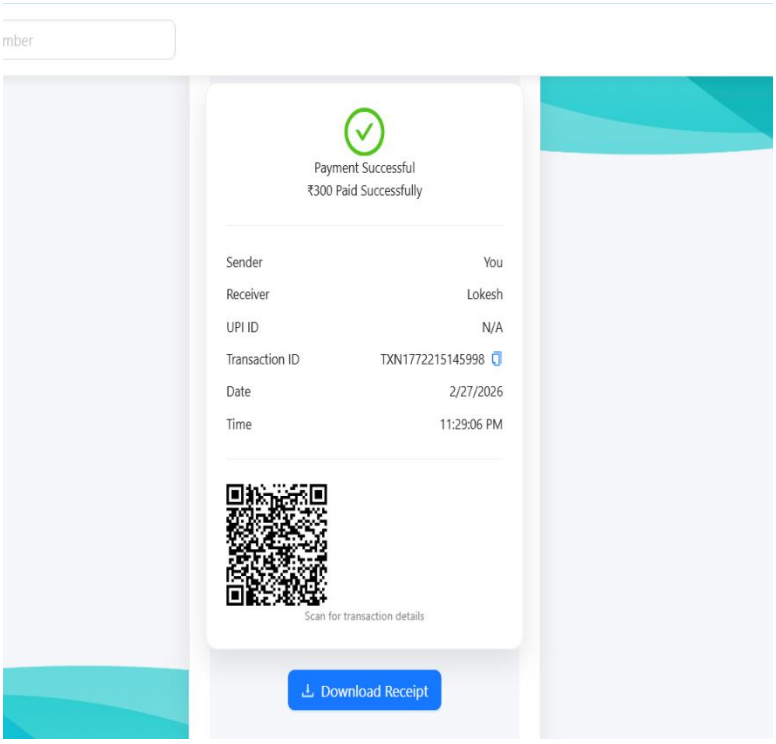

Figure 2 : OUTPUT DESIGN

4.3 DATABASE DESIGN

The Database Design of the HealPay system is implemented using MongoDB, a NoSQL document-based database that stores data in JSON-like format. The database is designed to efficiently manage user information and transaction records while maintaining high security and scalability. The primary collections in the system include the Users collection and the Transactions collection.

The Users collection stores details such as user ID, name, mobile number, encrypted password, wallet balance, and account creation date. Passwords are stored in encrypted form to prevent unauthorized access. The Transactions collection maintains records of each financial transaction, including transaction ID, sender ID, receiver ID, amount, timestamp, transaction status, fraud detection flag, and digital signature.

The database ensures that all transactions are securely recorded and easily retrievable for transaction history display. Encryption keys are securely exchanged using CRYSTALS-Kyber, and transaction authenticity is verified using CRYSTALS-Dilithium. The flexible structure of MongoDB allows efficient handling of large volumes of transaction data while maintaining data consistency and integrity.

TABLE NAME : Users

TABLE DESCRIPTION : User Register Informations

Column Name	Data Type	Constraints
_id	ObjectId	Primary Key
Name	String	NOT NULL
Email	String	UNIQUE
Mobile	Number	UNIQUE, NOT NULL
Password	String	NOT NULL
Amount	Number	NOT NULL
PublicKey	String	NOT NULL
PrivateKey	String	NOT NULL, Hidden
Notifications	Array	Embedded
Date	Date	Auto

Table 1: USER REGISTER INFORMATIONS

TABLE NAME : Transactions

TABLE DESCRIPTION : User Transaction Informations

Field Name	Data Type	Constraints
_id	ObjectId	Primary Key
SenderMobile	String	NOT NULL
ReceiverMobile	String	NOT NULL
Amount	Number	NOT NULL
Status	String	Success / Failed
EncryptedData	String	Nullable
Signature	String	Nullable
kyberPublicKey	String	Nullable
dilithiumPublicKey	String	Nullable
Date	Date	Auto

Table 2 : USER TRANSACTION INFORMATIONS

4.4 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) illustrates how data moves within the HealPay system from input to processing and output. At the highest level (Level 0 DFD), the system is represented as a single process interacting with the user and the database. The user provides registration, login, or transaction details to the HealPay system, which processes the request and stores or retrieves data from the database before returning a response.

At Level 1 DFD, the system is divided into detailed processes such as User Registration, User Authentication, Transaction Processing, Fraud Detection, and Notification Management. When a user initiates a transaction, the backend first validates the input data, verifies the wallet balance, and encrypts the transaction using Post-Quantum Cryptography. The transaction details are then sent to the Machine Learning fraud detection module for anomaly analysis. If the transaction is approved, it is digitally signed and stored in the Transactions collection, and the wallet balance is updated. Finally, the system sends a confirmation message or fraud alert back to the user.

The DFD clearly shows the interaction between the frontend, backend, database, cryptographic module, and fraud detection module, ensuring transparent and secure data movement throughout the system.

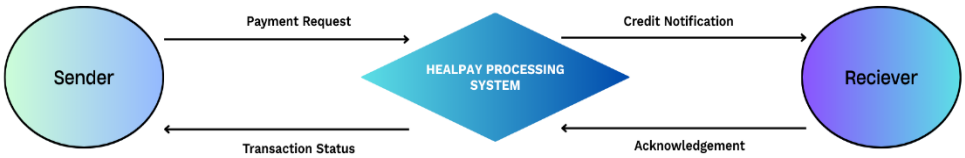

Figure 3 : HEALPAY PROCESS SYSTEM - LEVEL 0

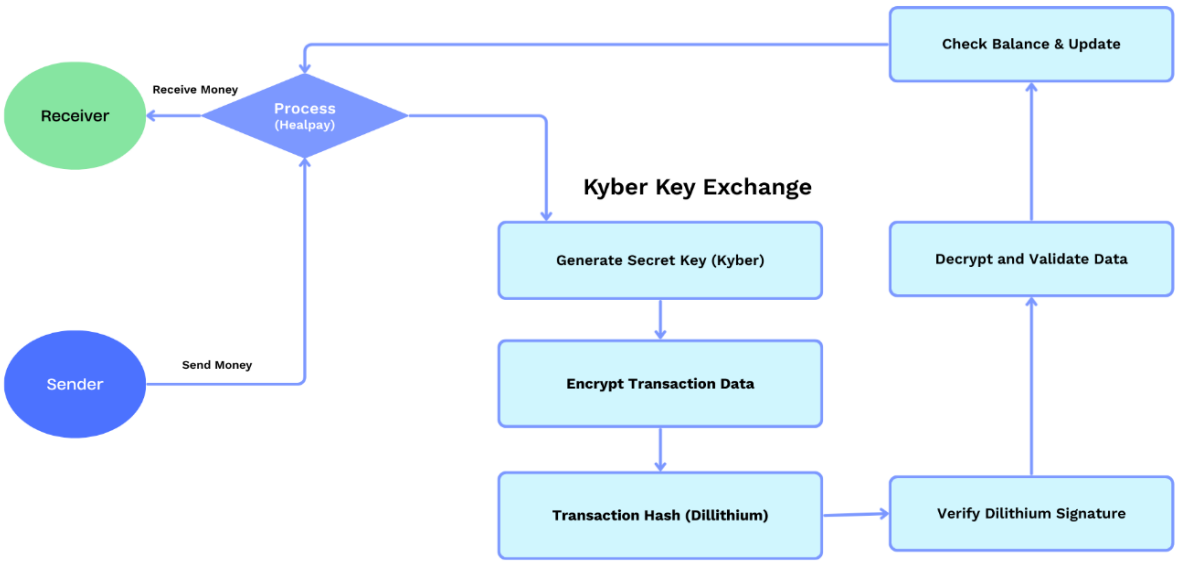

Figure 4 : HEALPAY PROCESS SYSTEM - LEVEL 1

4.5 ER DIAGRAM

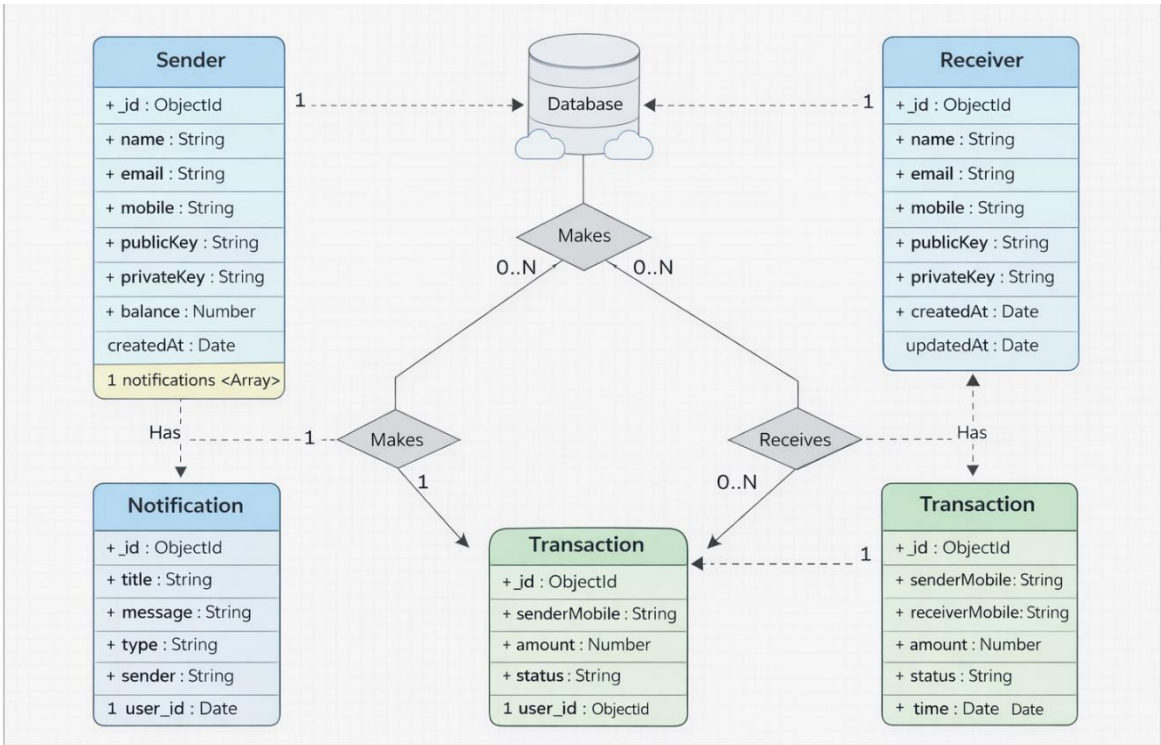

Figure 5 : ER DIAGRAM

5. SYSTEM TESTING AND IMPLEMENTATION

5.1 SYSTEM TESTING

System Testing is a crucial phase in the development of the HealPay – Smart Wallet with Embezzlement Analytics & Quantum Safe Cryptography system. It ensures that all modules function correctly, securely, and efficiently before deployment. The testing process verifies that the system meets functional requirements, security standards, and performance expectations. Based on the project implementation, different levels of testing were conducted to ensure reliability and robustness.

Unit Testing

Unit Testing was performed to test individual components of the system separately. Each module such as user login, balance calculation, transaction validation, and fraud detection logic was tested independently to ensure correct functionality without dependency on other modules. For example, the login module was tested by providing valid and invalid credentials to verify proper authentication behavior. Similarly, the balance calculation logic was tested to ensure that transactions are blocked when the available wallet balance is insufficient. This testing ensures that each function works correctly at the micro level.

Integration Testing

Integration Testing was conducted to verify the interaction between different modules of the system. Since HealPay follows a modular architecture consisting of React frontend, Node.js backend, MongoDB database, and Python microservices, it was essential to test how these components communicate with each other.

For example, the integration between the User Module and Transaction Module was tested to ensure that after successful login, users can perform transactions seamlessly. The backend API communication with the Machine Learning fraud detection service was also tested to confirm that transaction data is correctly analyzed before approval. This testing ensures smooth data flow across frontend, backend, database, and cryptographic modules.

Security Testing

Security Testing was a major focus of the project due to the sensitive nature of financial transactions. The system was tested against various types of cyber threats such as:

- Brute-force login attempts

- Phishing simulations
- Fake transaction injection
- Unauthorized API access

The authentication mechanism was tested by validating secure token generation and verification. Encrypted communication and digital signature verification were tested using Post-Quantum Cryptography algorithms such as CRYSTALS-Kyber for key exchange and CRYSTALS-Dilithium for digital signatures. These tests ensured that transactions remain confidential and tamper-proof.Security testing confirms that the system can resist unauthorized access and protect user data effectively.

API Testing

API Testing was performed using Postman to validate backend services. The Login API, Transaction API, and Fraud Detection API were tested for reliability, correctness, and response time. Various scenarios such as valid requests, invalid inputs, and error handling were verified. This ensured that APIs respond accurately and handle exceptions properly without system crashes.

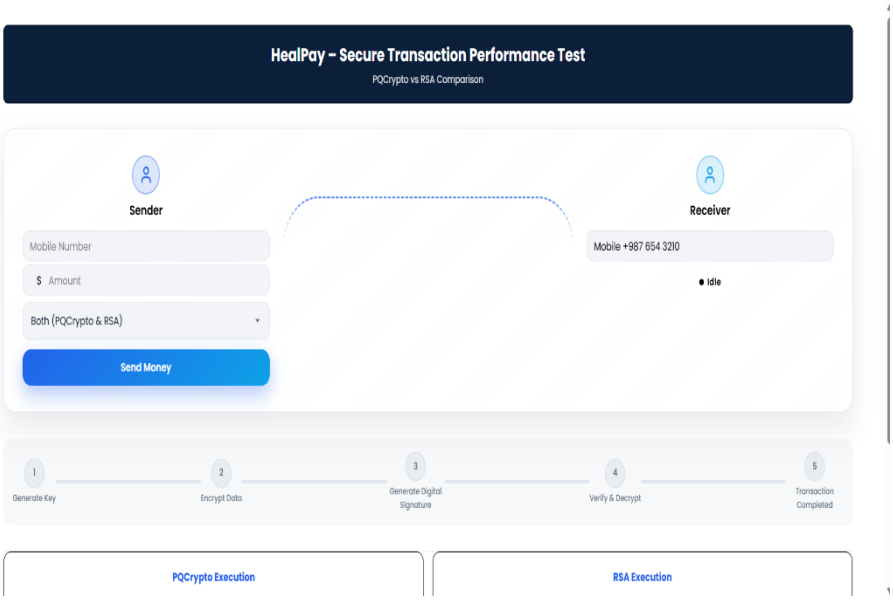

Figure 6 : TRANSACTION PERFORMANCE TESTING

5.2 SYSTEM IMPLEMENTATION

System Implementation refers to the practical execution of the designed HealPay – Smart Wallet with Embezzlement Analytics & Quantum Safe Cryptography system in a real-time working environment. This phase involves integrating all modules such as frontend interface, backend services, database management, Post-Quantum Cryptography, and Machine Learning-based fraud detection into a unified and fully functional application. The implementation ensures that the system operates according to the design specifications while maintaining high security, accuracy, and performance.

The HealPay system is implemented using a full-stack architecture. The frontend is developed using React.js, which provides a responsive and interactive user interface. The backend is developed using Node.js and Express.js, which handle server-side logic and API management. MongoDB is used as the database for storing user and transaction data. Python microservices are integrated for implementing Post-Quantum Cryptography and Machine Learning fraud detection.

The implementation begins with user registration and authentication. When a new user registers, their details such as name, mobile number, and password are collected and validated. The password is encrypted before storing it in the MongoDB database to ensure data confidentiality. During login, the system verifies the credentials and generates an authentication token to allow secure access to wallet services.

Once authenticated, the user can initiate a transaction. The implementation flow of a transaction is structured and secure. First, the user enters the receiver's mobile number. The backend validates whether the receiver exists in the Users collection. After successful validation, the user enters the transaction amount. The system checks the sender's wallet balance to ensure sufficient funds are available. If the balance is insufficient, the transaction is blocked immediately, preventing unauthorized overdraft.

If the balance is sufficient, the system proceeds to the transaction confirmation stage. This step ensures that the user intentionally approves the transaction before execution. After confirmation, the transaction details are processed through the cryptographic module. Secure key exchange is performed using CRYSTALS-Kyber, and the transaction is digitally signed using CRYSTALS-Dilithium to ensure authenticity, integrity, and protection against quantum computing attacks.

Following encryption and digital signing, the transaction data is forwarded to the Machine Learning fraud detection module. The ML model analyzes parameters such as transaction frequency, transaction amount variations, unusual login timing, and abnormal behavioral patterns. If the transaction is identified as suspicious, the system flags or blocks it and generates a fraud alert notification. If the transaction passes the fraud analysis, it is approved for execution.

Once approved, the system updates the sender's and receiver's wallet balances accordingly. The transaction details, including transaction ID, sender ID, receiver ID, amount, timestamp, fraud status, and digital signature, are stored securely in the Transactions collection in MongoDB. This ensures proper record maintenance and future retrieval for transaction history.

After successful execution, the system generates a payment receipt containing transaction details and confirmation status. The frontend dynamically updates the wallet balance and displays a success message such as "Money Sent Successfully" or "Money Received." Real-time notifications are provided to enhance user experience and transparency.

The modular microservice-based implementation ensures that each component functions independently while maintaining secure communication between modules using JSON-based API requests and responses. This architecture improves scalability, maintainability, and future enhancement capability.

6. CONCLUSION

The HealPay – Smart Wallet with Embezzlement Analytics & Quantum Safe Cryptography project successfully demonstrates the design and development of a secure, intelligent, and future-ready digital wallet system. The primary objective of this project was to enhance the security of online financial transactions by integrating Post-Quantum Cryptography and Machine Learning-based fraud detection into a modern digital payment platform.

Traditional digital wallet systems mainly rely on classical encryption techniques, which may become vulnerable with the advancement of quantum computing technologies. To overcome this limitation, the proposed system implemented quantum-resistant cryptographic algorithms such as CRYSTALS-Kyber for secure key exchange and CRYSTALS-Dilithium for digital signature verification. These algorithms ensure long-term data protection against both classical and future quantum-based cyber threats.

In addition to strong encryption, the system integrates a Machine Learning model capable of analyzing transaction behavior and detecting suspicious activities in real time. Unlike traditional rule-based fraud detection systems, the proposed intelligent model adapts to new patterns and prevents fraudulent transactions before completion. This proactive approach significantly improves system reliability and reduces financial risks.

The project also successfully implemented a modular full-stack architecture using React for frontend development, Node.js for backend services, MongoDB for secure data storage, and Python microservices for cryptographic and analytical operations. The inclusion of a hacker simulation module further strengthened the system by allowing controlled testing against brute-force attacks, phishing attempts, and unauthorized transaction injections.

Through systematic testing, secure implementation, and proper integration of all modules, the HealPay system achieved high levels of security, scalability, and performance. The system ensures secure authentication, encrypted transaction processing, real-time fraud alerts, structured transaction history management, and digital receipt generation.

In conclusion, the HealPay project provides a comprehensive solution for secure digital financial transactions by combining quantum-safe cryptography, intelligent fraud detection, and modern web technologies. The system not only addresses current cybersecurity challenges but also prepares the financial ecosystem for future technological advancements.

7. REFERENCES

[1] NIST, "Report on Post-Quantum Cryptography — NIST Post-Quantum Cryptography Project Report," National Institute of Standards and Technology, 2022.

[2] "An Efficient Fraud Detection Model for Online Payment Systems Based on Machine Learning," *Journal of Information Security and Applications*, vol. 54, pp. 102–110, 2021.

[3] "Feature Engineering Strategies for Credit Card Fraud Detection," *Expert Systems with Applications*, vol. 165, pp. 113–125, 2020.

[4] "A Secure Digital Wallet Scheme Based on Blockchain and Machine Learning," *IEEE Access*, vol. 9, pp. 45678–45690, 2021.

[5] "Analysis of Fraud Detection Techniques in Financial Transactions," *IEEE Symposium on Security and Privacy Workshops*, pp. 210–218, 2019.

[6] "Post-Quantum Cryptography: An Emerging Security Requirement for IoT," *International Journal of Computer Networks and Communications*, vol. 12, no. 4, pp. 45–56, 2020.

[7] "Smart Wallet Security Using Machine Learning and Cryptography Techniques," *International Journal of Computer Applications*, vol. 178, no. 32, pp. 15–22, 2019.

[8] "Quantum-Resistant Public Key Cryptographic Algorithms," *Journal of Cryptographic Engineering*, vol. 11, no. 2, pp. 89–101, 2021.

[9] "A Survey on Machine Learning Techniques for Credit Card Fraud Detection," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 5, no. 6, pp. 920–935, 2021.

[10] "A Comprehensive Review of Blockchain Based Secure Digital Wallet Systems," *Journal of Network and Computer Applications*, vol. 170, pp. 102–118, 2020.

[11] "Real-Time Transaction Fraud Detection in Financial Systems Using Random Forest," *International Journal of Intelligent Systems and Applications*, vol. 13, no. 3, pp. 34–42, 2021.

[12] “Cryptographic Algorithm Performance on Post-Quantum Platforms,” *Journal of Information Security and Applications*, vol. 58, pp. 201–212, 2022.

[13] “Deep Learning Approach for Fraud Detection in Payment Systems,” *Computers & Security*, vol. 105, pp. 102–115, 2021.

[14] “Machine Learning-Based Fraud Detection System for Mobile Wallet Applications,” *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*, vol. 12, no. 1, pp. 66–79, 2020.

[15] “Post-Quantum Cryptography: Challenges and Solutions,” *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–35, 2022.

[16] “Design and Implementation of Secure Electronic Payment System Using Cryptography,” *International Journal of Computer Science and Network Security*, vol. 19, no. 5, pp. 145–152, 2019.

[17] “Comparative Study of Quantum Resistant Key Exchange Algorithms,” *International Journal of Advanced Research in Computer Science*, vol. 11, no. 2, pp. 88–97, 2020.

[18] “Blockchain and Smart Contract Based Secure Wallet System,” *International Journal of Information Security Science*, vol. 10, no. 4, pp. 200–210, 2021.

[19] “Adaptive Machine Learning Models for Detecting Financial Fraud,” *Journal of Financial Crime*, vol. 28, no. 3, pp. 750–765, 2021.

[20] “Evaluation of Post-Quantum Signature Algorithms for Secure Transactions,” *Journal of Cryptographic Engineering*, vol. 12, no. 1, pp. 33–47, 2022.

8. APPENDIX

a) SAMPLE CODING :

```
from fastapi import FastAPI

from pydantic import BaseModel

import json, os, base64, hashlib, hmac

from cryptography.hazmat.primitives.ciphers.aead import AESGCM

app = FastAPI()

# □ Shared secret (same in Node.js)

SECRET_KEY = hashlib.sha256(b"healpay_secret_key").digest()

class Transaction(BaseModel):

    sender: str

    receiver: str

    amount: float

@app.post("/secure-transaction")

def secure_transaction(tx: Transaction):

    data = json.dumps(tx.dict()).encode()

    nonce = os.urandom(12)

    aes = AESGCM(SECRET_KEY)

    ciphertext = aes.encrypt(nonce, data, None)

    encrypted_payload = base64.b64encode(nonce + ciphertext).decode()

    signature = hmac.new(

        SECRET_KEY,

        encrypted_payload.encode(),

        hashlib.sha256
```

```
    ).digest()
    return {
        "encrypted_data": encrypted_payload,
        "signature": base64.b64encode(signature).decode()
    }
import pqcrypto.dilithium
def dilithium_sign(message):
    keypair = pqcrypto.dilithium.keypair()
    sig = pqcrypto.dilithium.sign(message.encode(), keypair.secret_key)
    return sig.hex(), keypair.public_key.hex()
import pqcrypto.kyber
def kyber_encrypt(message):
    keypair = pqcrypto.kyber.keypair()
    cipher = pqcrypto.kyber.encrypt(message.encode(), keypair.public_key)
    return cipher.hex(), keypair.public_key.hex()
import { useState } from "react";
import {
  Input,Button,Avatar,Card,Steps,Result,
} from "antd";
import {
  UserOutlined,
  LockOutlined,
  CheckCircleOutlined,
} from "@ant-design/icons";
```

```
import {
  Phone,
  IndianRupee,
  ShieldCheck,
} from "lucide-react";
import { toast, ToastContainer } from "react-toastify";
import axios from "axios";
import "react-toastify/dist/ReactToastify.css";
import "../styles/PayAnyone.css";
import PaymentReceipt from "../pages/PaymentReceipt";
import { useLocation } from "react-router-dom";
import { useEffect } from "react";
function PayAnyone() {
  const [step, setStep] = useState(0);
  const [loading, setLoading] = useState(false);
  const [success, setSuccess] = useState(false);
  const [showReceipt, setShowReceipt] = useState(false);

  const [receiver, setReceiver] = useState(null);
  const [balance] = useState(1000);
  const [txn, setTxn] = useState(null);
  const location = useLocation();

  const [data, setData] = useState({
```

```
    receiverMobile: "",
    amount: "",
    password: "",
  });

 useEffect(() => {
  if (location.state?.mobile) {
    autoCheckReceiver(location.state.mobile);
  }
}, [location.state]);

  const handleChange = (e) =>
    setData({ ...data, [e.target.name]: e.target.value });

  // STEP 1
  const checkReceiver = async () => {
    if (data.receiverMobile.length !== 10) {
      toast.error("Enter valid mobile number");
      return;
    }

    try {
      const token = localStorage.getItem("token");
```

```
    const res = await axios.get(
      `http://localhost:5000/api/profile/by-mobile/${data.receiverMobile}`,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      }
    );
    // □ REAL receiver data from DB
    setReceiver({
      name: res.data.name,
      mobile: res.data.mobile,
    });


    setStep(1);
  } catch (err) {
    toast.error(
      err?.response?.data?.message || "Receiver not found"
    );
  }
};
const autoCheckReceiver = async (mobile) => {
try {
  const token = localStorage.getItem("token");
```

```
    const res = await axios.get(
      `http://localhost:5000/api/profile/by-mobile/${mobile}`,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      }
    );
    setData((prev) => ({
      ...prev,
      receiverMobile: mobile,
    }));
    setReceiver({
      name: res.data.name,
      mobile: res.data.mobile,
    });
    setStep(1); // move directly to amount step
  } catch (err) {
    toast.error(
      err?.response?.data?.message || "Receiver not found"
    );
  }
};
```

```
// STEP 2
const checkAmount = () => {
  if (!data.amount || data.amount <= 0) {
    toast.error("Enter valid amount");
    return;
  }

  if (Number(data.amount) > balance) {
    toast.error("Insufficient balance");
    return;
  }
  setStep(2);
};
const sendMoney = async () => {
  if (!data.password) {
    toast.error("Enter password");
    return;
  }

  setLoading(true);

  try {
    const token = localStorage.getItem("token");
    if (!token) {
```

```
      toast.error("Session expired. Please login again.");
      return;
    }

    const res = await axios.post(
      "http://localhost:5000/api/user/send",
      data,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      }
    );
    // □ SAVE TRANSACTION DATA
   setTxn({
sender: res.data.senderName || "You",
receiver: res.data.receiverName || receiver.name,
amount: res.data.amount || data.amount,
upiId: res.data.upiId || "N/A",
txnId: res.data.transactionId || `TXN${Date.now()}`,
date: res.data.createdAt
  ? new Date(res.data.createdAt).toLocaleDateString()
  : new Date().toLocaleDateString(),
time: res.data.createdAt
```

```
    ? new Date(res.data.createdAt).toLocaleTimeString()
    : new Date().toLocaleTimeString(),
});
      // □ SUCCESS TOAST
      toast.success("Payment successful □");
      // □ Show success UI
      setSuccess(true);
      setShowReceipt(true);


    } catch (err) {
      toast.error(
        err?.response?.data?.message || "Transaction failed"
      );
    } finally {
      setLoading(false);
    }
  };
  return (
    <div className="pay-container">
      <ToastContainer />
<div className="pay-title">SEND MONEY</div>
      <Card className="pay-card">
        {/* Stepper */}
        <Steps
```

```
  current={step}
  items={[
    { title: "Mobile", icon: <Phone size={18} /> },
    { title: "Amount", icon: <IndianRupee size={18} /> },
    { title: "Confirm", icon: <ShieldCheck size={18} /> },
  ]}
/>

{/* SUCCESS */}
{success && receiver && (
  <Result
    status="success"
    icon={<CheckCircleOutlined className="success-icon" />}
    title="Payment Successful"
    subTitle={`₹${data.amount} sent to ${receiver.name}`}

    extra={[
      <Button
        type="primary"
        key="receipt"
        onClick={() => setShowReceipt(true)}
      >
        View Receipt
      </Button>,
```

```
      <Button
        key="again"
        onClick={() => window.location.reload()}
      >
        Pay Again
      </Button>,
    ]}

  />
)}
{showReceipt && txn && (
  <PaymentReceipt
    sender={txn.sender}
    receiver={txn.receiver}
    amount={txn.amount}
    upiId={txn.upiId}
    txnId={txn.txnId}
    date={txn.date}
    time={txn.time}
  />
)}
{/* STEP 1 */}
    <Button
      type="text"
```

```
            disabled={loading}
            onClick={() => setStep(1)}
          >
            ← Back
          </Button>

          <Button
            type="primary"
            size="large"
            block
            loading={loading}
            onClick={sendMoney}
          >
            Send ₹{data.amount}
          </Button>
        </div>
      )}
    </Card>
  </div>
);
}
export default PayAnyone;
```

b) SCREENSHOTS :

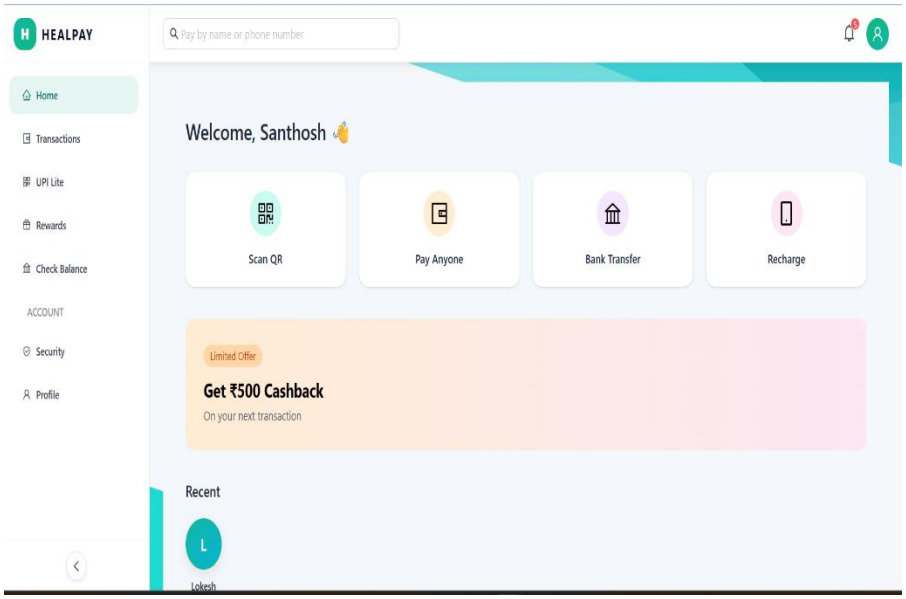

Figure 7 : HOME PAGE DESIGN

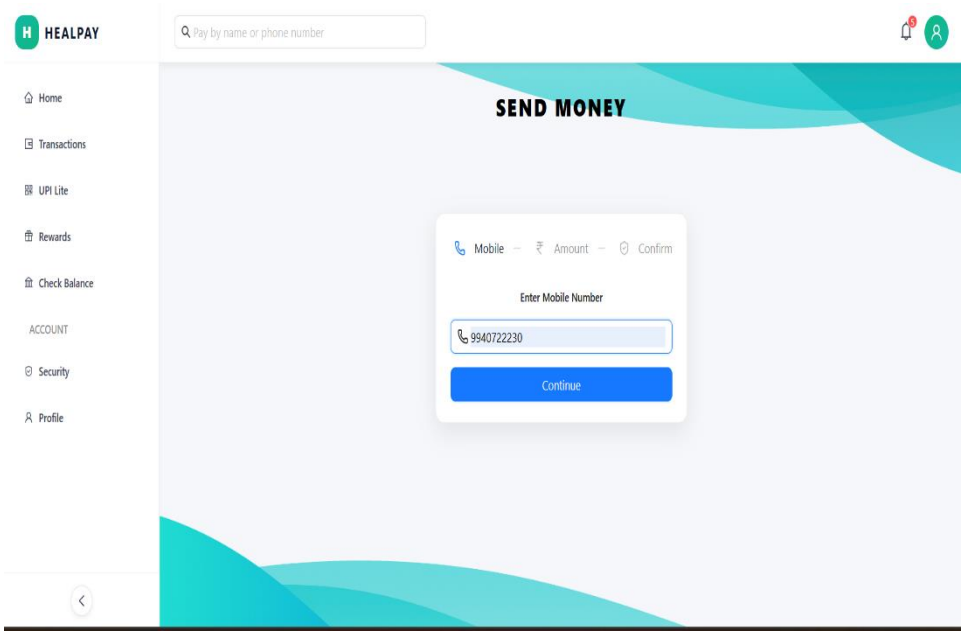

Figure 8 : PAYMENT PAGE DESIGN

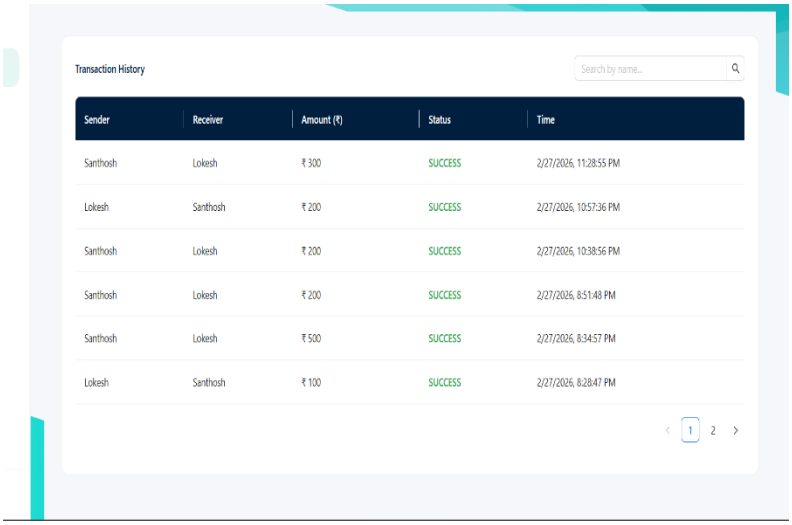

Figure 9 : TRANSACTION HISTORY PAGE DESIGN

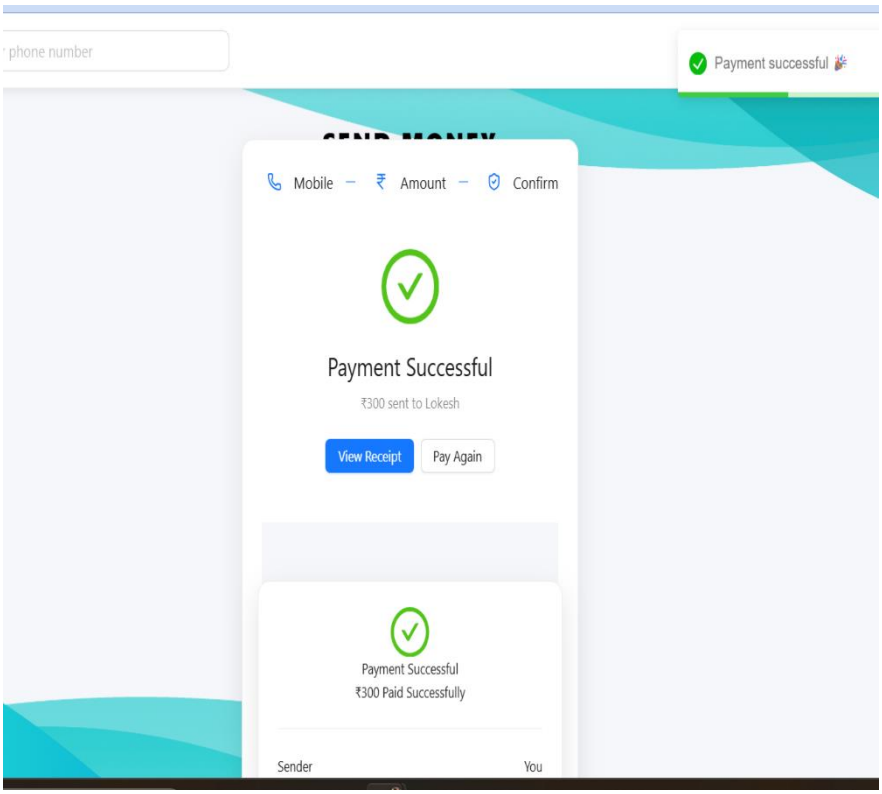

Figure 10 : PAYMENT PAGE DESIGN

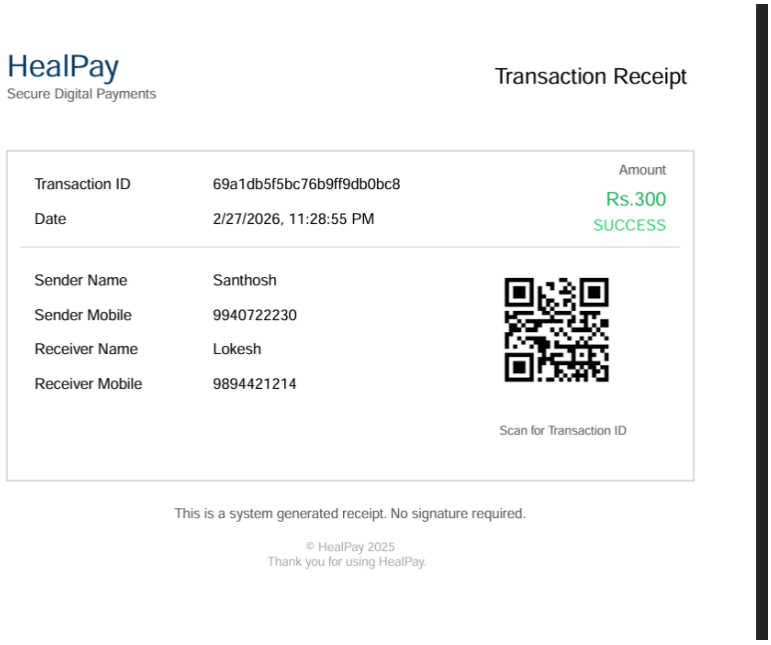

Figure 11 : TRANSACTION RECEIPT DESIGN

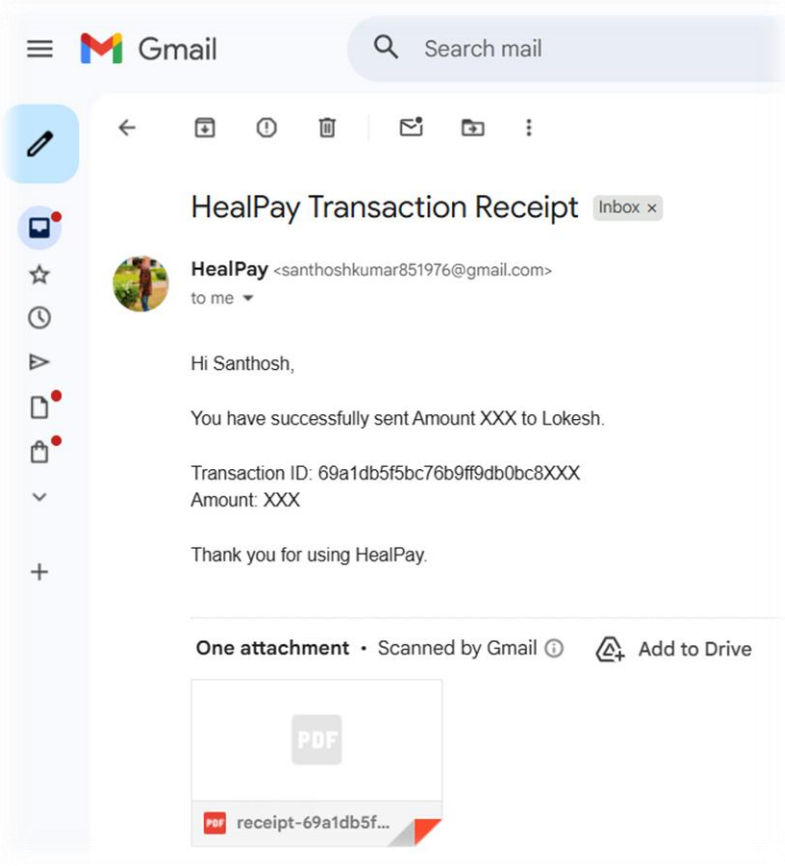

Figure 12 : EMAIL MESSAGE DESIGN